POSTGUARD DESKTOP

System Workflow & Implementation Summary

สรุปการทำงานของระบบ วิธีที่ใช้ และการไหลของข้อมูลตั้งแต่ผู้ใช้เข้าสู่ระบบจนถึงรายงานผล

Desktop App | FastAPI | Supabase PostgreSQL | MediaPipe | Camera Adjustment | Personal Baseline | RULA-adapted | Risk Events

---

โครงการ: PostGuard
Branch: feature/detection
สถานะเอกสาร: อ้างอิงโค้ดปัจจุบัน
วันที่: 27 ตุลาคม 2026

1. ภาพรวมระบบ

PostGuard เป็น Desktop Application สำหรับประเมินท่าทางจากกล้องด้านหน้า ภาพกล้องถูกประมวลผลบนเครื่องผู้ใช้ ส่วน Backend รับผิดชอบบัญชี สิทธิ์ ข้อมูล Session, Baseline, Risk Event, Alert และ Report

สมัคร/เข้าสู่ระบบ → ตรวจ Privacy Consent → Dashboard
เลือกกล้อง → Camera Adjustment → Corrected Preview
Personal Baseline → Monitoring → Risk Event / Alert
Stop → Session Summary → Dashboard / History / Report

Desktop
PySide6 UI, OpenCV, MediaPipe, การประมวลผลท่าทาง, เสียงและ popup

Backend
FastAPI, JWT, validation, ownership checks และ REST API

Database
Supabase PostgreSQL ผ่าน SQLAlchemy และ Alembic migrations

External services
Resend สำหรับ OTP และ private object storage สำหรับ avatar

หลักการแบ่งความรับผิดชอบ

ชั้น	หน้าที่	ข้อมูลสำคัญ
UI Pages / Dialogs	รับคำสั่งและแสดงสถานะให้ผู้ใช้	ไม่คำนวณ risk policy เอง
Desktop Services	เชื่อม UI กับ Vision และ API	จัดการ camera, persistence, session
Vision Pipeline	ตรวจ landmark, สร้าง measurement, assessment และ risk	ทำงานในเครื่องผู้ใช้
Backend	ตรวจสิทธิ์และบันทึกข้อมูลถาวร	ไม่รับภาพกล้องดิบ

2. Authentication, Privacy และข้อมูลผู้ใช้

Registration และ OTP

กรอกข้อมูล + @ku.th + Confirm Password → ส่ง OTP ผ่าน Resend
→ Verify OTP → Hash Password → สร้างบัญชี

- ระบบรับอีเมลมหาวิทยาลัย **@ku.th** ตามขอบเขตโครงการ
- Password ถูก hash ก่อนจัดเก็บ และไม่ส่งกลับไปยัง Desktop
- Login สำเร็จได้ JWT; Desktop แนบ Bearer token กับ API ส่วนตัว
- Forgot Password ใช้ OTP → reset token อายุจำกัด → ตั้งรหัสผ่านใหม่

Privacy Consent

ก่อนเปิด Camera Flow ระบบอ่าน consent version ปัจจุบัน หากยังไม่ยินยอมจะแสดง Privacy Notice และบันทึกเวลา/เวอร์ชันเมื่อยอมรับ Backend ตรวจซ้ำก่อนสร้าง Session เพื่อป้องกันการข้ามขั้นจาก client

Profile, Settings และ Account Lifecycle

ส่วน	วิธีทำงาน
Profile	แก้ username/full name และยืนยันก่อน PATCH; avatar ใช้ private storage และ endpoint ที่ตรวจ JWT
Settings	บันทึกเปิด/ปิด notification, sound, volume และ reminder interval ต่อผู้ใช้
Delete account	ขอคำยืนยัน → ตั้งสถานะ PENDING_DELETION → มีช่วงกู้คืน 14 วัน → background purge ตรวจทุก 6 ชั่วโมง
Ownership	ทุก Camera, Baseline, Session, Risk และ Report ถูกกรองด้วย user จาก JWT

Privacy by design: ภาพกล้องและภาพบัตรอ้างอิงไม่ถูกอัปโหลด Backend เก็บเฉพาะค่าตัวเลขและผลสรุปที่จำเป็น

3. Camera Flow และการแก้ภาพเอียง

1. Scan & Select → 2. Camera Adjustment
→ 3. Corrected Preview → 4. Personal Baseline

Camera Scan

1. OpenCV ทดลองเปิด device index และอ่าน frame จริง
2. แสดงเฉพาะกล้องที่พร้อมใช้งาน พร้อมชื่อและ resolution
3. กล้องที่เลือกถูกลงทะเบียนด้วย device identifier เช่น `opencv-index:0`

Camera Adjustment ด้วยบัตรอ้างอิง

ขั้นตอน	วิธี
ค้นหาบัตร	Grayscale → Gaussian blur → Canny edge → dilation → contour/hull → polygon 4 มุม
ตรวจรูปทรง	อัตราส่วนบัตรมาตรฐาน 85.60 × 53.98 มม., มุมใกล้ 90°, ตำแหน่งและขนาดอยู่ใน guide
ตรวจแนว	คำนวณ card roll และ perspective difference; roll ต้องไม่เกิน ±5° และ perspective ไม่เกิน 10%
ความนิ่ง	ต้องผ่าน 10 เฟรมติดต่อกัน พร้อมตัวนับ 1/10–10/10
ผลลัพธ์	resolution, pixel scale X/Y/average, card roll, perspective และเวลาที่ตรวจ

Corrected Preview

หลัง Adjustment ระบบนำ `card_roll_deg` มาสร้าง affine rotation matrix ด้วย OpenCV แล้วใช้ `warpAffine` หมุนชดเชยภาพ โดยรักษาขนาด frame เดิมและใช้ border replication ลดขอบดำ

Corrected Frame = warpAffine(Raw Frame, rotation(card_roll_deg))

เฟรมหลังแก้เอียงถูกใช้ทั้ง Preview, Pose/Face Detection, Personal Baseline และ Monitoring ทำให้สิ่งที่ผู้ใช้เห็นตรงกับข้อมูลที่ระบบนำไปคำนวณ

4. Personal Baseline

> Baseline คือท่านั่งปกติของผู้ใช้แต่ละคน ใช้ลดปัญหาการตัดสินทุกคนด้วยมุมคงที่เดียวกัน และผูกกับกล้อง/การตั้งกล้องครั้งนั้น

Corrected Frame → Face + Pose valid
→ เก็บ Sample → ตรวจเวลา/สัดส่วน/ความนิ่ง
→ Candidate → ผู้ใช้ยืนยัน → Save Baseline

ค่าที่เก็บ	การใช้งาน
Shoulder tilt	ระดับไหล่ซ้าย–ขวาปกติ
Neck lateral tilt	การเอียงคอด้านข้างปกติ
Neck flexion	ค่ากัม/เงยคออ้างอิง
Eye distance proxy	บริบทระยะผู้ใช้กับกล้อง
Camera reference	scale, roll, perspective, frame size และ calibrated time

การตรวจคุณภาพ

- นับเฉพาะเฟรมที่มีข้อมูลจำเป็นครบและเป็นค่าจำนวน finite
- ตรวจจำนวน sample, เวลาครอบคลุม, valid ratio และ stability
- เมื่อผ่านจึงสร้าง Candidate; ผู้ใช้ต้องยืนยันก่อนบันทึก
- Start Monitoring เปิดใช้งานหลัง Backend ตอบกลับ baseline_id สำเร็จ

การเปรียบเทียบขณะ Monitoring ใช้ผลต่างจาก Baseline ไม่ได้ใช้มุมกล้องดิบเพียงค่าเดียว จึงรองรับความแตกต่างด้านสรีระและท่านั่งเริ่มต้นได้ดีกว่า

5. Detection Pipeline

Raw Frame → Roll Correction → MediaPipe
→ Feature Validation → Posture Assessment
→ Temporal Tracking → Risk Evaluation

1. **Capture:** QTimer อ่านกล้องประมาณทุก 30 ms; detection ทำทุก 5 เฟรมเพื่อลดภาระ
2. **MediaPipe:** Pose Landmarker และ Face Landmarker สร้าง landmark บนเครื่อง
3. **Context filter:** ตรวจว่ามีใบหน้า/ลำตัวและบริบทเหมาะสมก่อนอนุญาตให้ประเมิน
4. **Features:** คำนวณ shoulder tilt, neck lateral tilt, neck flexion และ eye state/blink
5. **Baseline comparison:** เทียบค่าปัจจุบันกับค่าปกติส่วนบุคคลและ threshold
6. **Assessment:** สร้าง posture factors เฉพาะค่าที่ตรวจได้จากกล้องด้านหน้า
7. **Temporal tracking:** ติดตามว่า factor ต่อเนื่องกี่วินาที; data gap ไม่ถูกนับเป็นหลักฐานใหม่
8. **Risk:** คำนวณ RULA-adapted และส่งต่อ Risk Event Tracker

กรณีข้อมูลไม่ครบ

หาก landmark หายหรือ feature ไม่ valid ระบบส่งสถานะ `NOT_EVALUATED` / `DATA_GAP` และแสดง N/A แทนการสร้างคะแนนจากข้อมูลเดา

6. RULA-adapted Risk Score

> ระบบประยุกต์แนวคิดจาก RULA เฉพาะส่วนที่กล้องด้านหน้าตรวจได้ จึงเรียกว่า **RULA-adapted Front Camera** ไม่ใช่ Full RULA และไม่ใช่การวินิจฉัยทางการแพทย์

หลักฐาน	เงื่อนไข	คะแนนภายใน
Neck flexion	0–10°	Base 1
Neck flexion	>10–20°	Base 2
Neck flexion	>20°	Base 3
Neck extension	ค่าต่ำกว่า 0°	Base 4
Neck side bending	เกิน tolerance จาก baseline	+1
Shoulder asymmetry	เกิน tolerance จาก baseline	+1
Sustained posture	factor ต่อเนื่องมากกว่า 60 วินาที	+1

Internal Score = min(7, Neck Base + Side Bend + Shoulder Asymmetry + Sustained Posture)

คะแนนภายใน	ระดับ	เปอร์เซ็นต์ที่แสดง
1–2	LOW	0–16.7%
3–4	MODERATE	33.3–50%
5–6	HIGH	66.7–83.3%
7	VERY HIGH	100%

Display Percentage = (Internal Score − 1) ÷ 6 × 100

Database และ Session Summary ยังเก็บคะแนน 1–7 เพื่อรักษาความหมายของ model และข้อมูลย้อนหลัง ส่วน Monitoring, Dashboard, Graph และ Report แปลงเป็น 0–100% ในชั้นแสดงผล

7. Risk Event และ Alert Policy

Risk Result → Risk Event Tracker
→ START / UPDATE / ESCALATE / RESOLVE
→ Alert Policy → Popup + Sound + Database

Risk Event Tracker

รวมเฟรมต่อเนื่องให้เป็นเหตุการณ์เดียว เพื่อลดการสร้าง Alert ทุกเฟรม รองรับสถานะ STARTED, UPDATED, ESCALATED, DEESCALATED, DATA_GAP และ RESOLVED พร้อม runtime event id

Alert Policy

การตัดสินใจ	เงื่อนไข
FIRST	แจ้งครั้งแรกเมื่อ event ระดับ MODERATE/HIGH ผ่าน first delay
REPEAT	เตือนซ้ำเมื่อพ้น cooldown; MODERATE 600 วินาที, HIGH 300 วินาที
ESCALATION	ระดับสูงขึ้นสามารถข้าม cooldown
SUPPRESSED	event ยังทำงานแต่ยังไม่ถึงเวลาที่ควรแจ้งซ้ำ

เมื่อ Policy อนุญาต Desktop แสดง popup แบบ non-modal และเล่นเสียงตาม Settings จึงไม่หยุด camera timer ผู้ใช้ยังทำงานต่อได้

การบันทึกแบบไม่บล็อก UI

- `RiskPersistenceService` ใช้ worker thread เดียวเรียงลำดับคำขอ
- lifecycle transition และ notification ถูกส่งทันที; UPDATE ปกติ throttle ทุก 5 วินาที
- retry สูงสุด 3 ครั้ง และใช้ idempotency key ป้องกันข้อมูลซ้ำ
- ก่อน Pause/Stop ระบบ flush งานที่ค้างเพื่อไม่ให้ event สูญหาย

8. Session Lifecycle และ Recovery

Start → RUNNING ⇄ PAUSED
Stop → COMPLETED → Summary Saved
ปิดผิดปกติ → Recovery หลัง Login → INTERRUPTED

สถานะ	ความหมาย	พฤติกรรม
RUNNING	กำลังตรวจจับ	timer, camera และ risk pipeline ทำงาน
PAUSED	พักชั่วคราว	ไม่นับช่วงพักเป็น active time และปิด active risk event
COMPLETED	ผู้ใช้ Stop สำเร็จ	บันทึก duration, summary และ report
INTERRUPTED	session ค้างจากการปิดผิดปกติ	กู้สถานะเมื่อ Login ครั้งถัดไปโดยไม่สร้าง Summary เทียม

การเปลี่ยนหน้า

เมื่อผู้ใช้ไป Dashboard หรือ History ระหว่าง RUNNING การตรวจจับยังทำงานด้วย timer เดิม ปุ่ม Monitoring จะพากลับ session เดิม ส่วน global Pause/Stop ทำงานได้จาก navigation area

การจบ Session

1. flush Risk Event/Notification ที่รอบันทึก
2. เรียก End Session และรับ duration จาก Backend
3. คำนวณสถิติจาก sample ในหน่วยความจำ
4. บันทึก Session Summary หนึ่งรายการต่อ Session
5. หากบันทึกล้มเหลว เก็บ snapshot เพื่อ Retry และไม่เริ่ม session ใหม่ทับข้อมูล

9. Dashboard, History และ Report

Dashboard

- จำนวน completed, active/interrupted และ active monitoring time
- ค่าเฉลี่ยและค่าสูงสุด Risk แสดงเป็นเปอร์เซ็นต์
- ตาราง Session ล่าสุดและจำนวน Alert ที่บันทึกจริง
- กราฟ RULA Score Trend เรียงตามเวลา: สีน้ำเงิน = ค่าเฉลี่ย, สีแดง = ค่าสูงสุด
- โหลด API ผ่าน Qt thread pool เพื่อไม่บล็อกการตรวจจับที่กำลังทำงาน

History และ Saved Report

History โหลด Sessions และ Summaries ที่เป็นของผู้ใช้ เลือก session เพื่อเปิดรายงานที่บันทึกไว้ รายงานอ่านค่าจากฐานข้อมูลและไม่คำนวณ detection ใหม่ จึงตรวจสอบย้อนหลังได้ตรงกับเวลาที่จบ session

Alert Events

แสดง Risk Event และ Alert Notification แยกกัน ผู้ใช้เห็นเวลาเริ่ม/จบ ระยะเวลา ระดับสูงสุด ปัจจัย และชนิดการแจ้งเตือน ทำให้ตรวจสอบได้ว่า posture risk เกิดขึ้นเมื่อใดและระบบแจ้งผู้ใช้จริงเมื่อใด

> ค่า N/A มีความหมายว่าไม่มีข้อมูลที่ประเมินได้หรือยังไม่ได้เชื่อม calculation นั้น ระบบไม่แทนข้อมูลที่หายด้วยศูนย์และไม่สร้างคะแนนสมมติ

10. Data Model และ API Flow

Desktop UI → APIClient + JWT → FastAPI Router
→ Service / Validation → SQLAlchemy → PostgreSQL

Entity	ข้อมูลหลัก	ความสัมพันธ์
User	identity, profile, consent, account lifecycle	เจ้าของข้อมูลทั้งหมด
Camera	device id, name, resolution	User 1:N Camera
PersonalBaseline	posture reference + camera calibration	User/Camera 1:N Baseline
Session	status, start/end, pause, durations	ผูก User + Camera + Baseline
RiskEvent	lifecycle, level, factors, duration	Session 1:N RiskEvent
AlertNotification	FIRST/REPEAT/ESCALATION, emitted time	RiskEvent 1:N Notification
SessionSummary	average/max risk, counts, angles, blink	Session 1:1 Summary

ความถูกต้องของข้อมูล

- Foreign key และ cascade delete รักษาความสัมพันธ์
- Unique constraint ของ runtime event/notification ทำให้ retry ไม่สร้างแถวซ้ำ
- Pydantic schemas ตรวจ request/response; Backend ตรวจ session ownership ซ้ำ
- Alembic ใช้จัดลำดับ schema migration สำหรับ deployment

11. Tech Stack และโครงสร้างโค้ด

ส่วน	เทคโนโลยี	เหตุผล
Desktop UI	Python, PySide6/Qt	สร้าง native desktop UI, signal/slot, timer และ worker
Computer Vision	OpenCV, MediaPipe	อ่าน/ปรับภาพและตรวจ landmark แบบ local
Backend	FastAPI, Pydantic	REST API, validation และ dependency-based auth
Data	SQLAlchemy, Alembic, PostgreSQL/Supabase	ORM, migrations และฐานข้อมูลถาวร
Email	Resend + postguardapp.xyz	ส่ง Registration/Password Reset OTP
Testing	unittest, mocks, Qt offscreen	ทดสอบ flow โดยไม่ต้องใช้กล้องจริงทุกครั้ง

โครงสร้างหลัก

Path	หน้าที่
`desktop/app/ui`	pages, dialogs และ windows
`desktop/app/services`	API, camera adjustment, baseline, persistence
`desktop/app/vision`	detection, assessment, temporal, RULA, event, policy
`backend/app/routers`	HTTP endpoints
`backend/app/services`	business logic และ external integration
`backend/app/models|schemas`	database entities และ API contracts

12. การทดสอบและข้อจำกัด

ผลทดสอบล่าสุด

ชุดทดสอบ	ผล	ครอบคลุม
Desktop Unit Tests	**214 ผ่านทั้งหมด**	UI flow, camera adjustment, baseline, monitoring, risk, alert, dashboard, history
Camera correction	ผ่าน	หมุนแก้เส้นเอียงและรักษาขนาด frame
Corrected Preview contract	ผ่าน	ยืนยันว่า detector รับเฟรมหลัง adjustment
Hardware test	ต้องรันบนเครื่องที่เปิด webcam ได้	ไม่รวมใน unit suite

ข้อจำกัดที่ต้องสื่อสาร

- ใช้กล้องด้านหน้าเพียงตัวเดียว จึงไม่เห็นข้อมือ แขน การบิดคอ แรง และภาระงานครบแบบ Full RULA
- Shoulder asymmetry เป็นการประยุกต์ของ PostGuard ไม่ใช่ item อย่างเป็นทางการของ RULA
- Risk percentage คือการแปลงเชิงเส้นเพื่อความเข้าใจง่าย ไม่ใช่ความน่าจะเป็นของการบาดเจ็บ
- ผลลัพธ์เป็นเครื่องมือคัดกรอง/เตือนท่าทาง ไม่ใช่คำวินิจฉัยหรือคำแนะนำทางการแพทย์
- ควรทดสอบ webcam, แสง, ระยะ, CPU และ session ระยะยาวบนเครื่องเป้าหมายก่อน release

การตีความ: คะแนนสูงหมายถึงหลักฐานท่าทางที่ระบบรองรับมีความเสี่ยงมากขึ้น ผู้ใช้ควรปรับท่าและพักตามความเหมาะสม แต่ไม่ควรใช้แทนผู้เชี่ยวชาญด้านการยศาสตร์หรือแพทย์

สรุป

PostGuard ใช้การประมวลผลภาพแบบ local ผสาน Personal Baseline, temporal evidence, RULA-adapted scoring และ event-based alerting แล้วบันทึกเฉพาะข้อมูลที่จำเป็นผ่าน Backend ที่ตรวจสิทธิ์ ทำให้ผู้ใช้ได้รับ feedback แบบ real time และตรวจสอบแนวโน้มย้อนหลังได้โดยไม่อัปโหลดภาพกล้อง